# Sci-Pal 核心数据流图 — 从数据下载到综述生成

6 个阶段，完整调用链路，展示每个模块的输入输出和调用关系

## 阶段 1：数据采集与入库（离线，只改数据层）

1. **download_pubmed_abstracts.py** → PubMed API 搜索+下载 → abstracts_xml/*.json
2. **download_pmc_xml.py / download_pmc_v2.py** → PMC OA API 下载全文 XML → papers_xml/*.xml
3. **download_openalex.py** → OpenAlex API（2亿+免费） → data/openalex_*.jsonl
4. import_xml.py / import_xml_v2.py → 解析 XML + 提取段落 → research.db（全文 SQLite）
5. **build_abstracts_db.py** → 加载所有 JSON + bge-base 生成向量 → abstracts.db + chroma_abstracts/ + FTS5索引
6. build_vector_index.py / build_paragraph_vector_index.py → 全文段落分块 + 生成向量 → chroma/（全文向量库）

核心原则：只更新数据库/向量库，检索层和主程序代码一行都不用改！新数据自动被检索覆盖。

▼▼▼

## 阶段 2：大纲加载与编辑（前端交互）

1. **outline_editor.js · 选择模板** → OUTLINE_TEMPLATES（Systematic/Narrative/Methodological）
2. **outline_editor.js · 加载已保存大纲** → GET /api/review/outline/saved/{name} → outline_store.py 读取
3. **outline_core.js · 编辑大纲** → 增删章节/段落 + 编辑关键词（keywords字段）
4. 保存大纲 → POST /api/review/outline/saved → outline_store.py 持久化

V32.1 改造：去掉了"自动生成大纲"按钮，改用预设模板 + 可编辑大纲，用户完全掌控结构。

▼▼▼

## 阶段 3：按大纲搜索并分配论文（核心检索流程）

1. **前端点击"按大纲搜索并分配"** → POST /api/review/search_and_assign → **review_generator.search_and_assign_papers()**
2. V32.4 主题核心关键词自动注入 → 从综述主题提取核心词（去停用词）→ 加到每章搜索词最前面
3. 逐章节搜索（for each section） → **unified_retriever.retrieve(query=章节标题+关键词)**

检索内部流程（**unified_retriever**）：
query_expander（查询扩展）→ scispacy NER 提取实体 → Entity Boost（实体加权检索）→ hybrid_search（向量召回 chroma + FTS5 全文召回）→ reranker（bge-reranker-base 重排序）→ MMR diversifier（多样性去重）→ abstract_search（摘要库补充搜索）→ 返回 top N 段落

4. **evidence_extractor（简化证据抽取）** → 从检索到的段落提取 EvidenceCard → evidence_cards 列表
5. 分配到章节+段落 → assigned_evidence_ids 写入每个 section/paragraph
6. 总数限定（max_papers） → 按相关性排序，取 top N，不超过用户设定的最大论文数
7. 返回 custom_outline（含 sections + evidence_cards） → 前端保存到 currentOutline，显示论文列表

V31.82 ：综述生成完全跳过关系抽取和冲突检测（这些是问答功能用的，综述不需要）。

▼▼▼

## 阶段 4：综述生成（V32.5 三大升级核心）

1. **前端点击"生成综述（英文）"** → POST /api/review/generate（含 custom_outline） → **review_generator.generate()**
2. V31.83 检测 custom_outline.evidence_cards → 有证据卡片 → 跳过检索，直接用按大纲搜索分配的证据
3. **_write_draft() 逐章节生成循环**

每章节内部流程（V32.5 三大升级）：

**生成阶段（_write_section）**

a. 章节类型感知：判断 Intro/Methods/Results/Discussion/Conclusion → 不同写作风格指导

b. 渐进式上下文注入：传入 previous_sections_summary（前文摘要，最多800字）→ 避免重复

c. 证据-声明矩阵：先列3-5个核心要点，每个要点映射证据ID，再基于有证据支撑的要点写正文

d. llm_client → Ollama qwen2.5:7b 生成章节正文（200-400词）

**评审修订阶段（_review_section）**

e. 5维评分标准：相关性30% + 证据支撑30% + 连贯逻辑20% + 学术写作10% + 无无依据推测10%

f. LLM 当评审专家，打分 + 列出问题 + 修改建议

g. 分数 < 60 → 把评审建议作为 feedback 注入 prompt，重新生成（最多重试1次）

h. 分数 ≥ 60 → 通过，进入下一章节

4. _split_and_link() → 拆分句子 + 提取 [ev_xxx] 引用标记 + 关联证据ID
5. _auto_cite_sentences() → 自动补全缺失的引用（基于语义匹配证据卡片）
6. 更新前文摘要 → 截取当前章节前200字 → 追加到 previous_sections_summary（最多800字）
7. ReviewDraft（所有章节句子 + 引用关联）

▼▼▼

## 阶段 5：格式化与存储

1. **citation_formatter.py** → 引用格式化（APA / Nature / Vancouver 三种风格）
2. citation_verifier.py → 引用验证（检查引用ID是否有效、是否对应正确论文）
3. sentence_locator.py → 句子原文定位（点击引用跳转到原文PDF对应位置）
4. review_store.py → 持久化存储综述（review_id + 完整内容 + 引用列表）
5. 返回 ReviewOutput（Markdown 全文 + 参考文献 + 统计 + 警告）

▼▼▼

## 阶段 6：前端展示与交互

1. **review.js 接收生成结果** → 渲染综述正文（三栏布局：大纲 | 正文 | PDF）
2. 点击正文引用标记 [1] → sentence_locator 定位 → pdf_viewer.js 打开对应论文PDF并滚动到位置
3. paper_list.js → 显示相关论文列表（标题/作者/期刊/年份/证据数）
4. pdf_annotator.js → PDF 标注（高亮/笔记，关联到综述句子）
5. supplementary_manager.js → 补充材料管理（表格/图表/数据集）
6. resizer.js → 三栏布局拖拽调整（大纲宽度 | 正文宽度 | PDF宽度）

右上角实时统计：句子数 / 证据数 / 通过率（引用验证通过率）

### 架构核心设计原则总结

1. **数据层与检索层解耦**：只更新数据库/向量库，检索层和主程序代码一行都不用改，新数据自动被覆盖。
2. **大纲先行 + 大纲引导检索**：用户完全掌控大纲结构，每章节用自己的关键词搜索，确保内容跟标题相关。
3. **证据-声明矩阵**：每个声明必须有证据支撑，杜绝 LLM 凭空编内容，所有引用可追溯到原文。
4. **迭代精炼 + 评分引导**：生成后自动评审打分，不达标就修订，确保输出质量。
5. **全本地部署**：Embedding + Reranker + LLM 全部本地运行，数据不出本机，无 API 费用。
6. **不用多智能体架构**：有精确数据库不需要推理协商，用确定性流程 + 评分标准更可控、更高效。